UNIVERSITY OF OKLAHOMA
GRADUATE COLLEGE

A CONFORMANCE HARNESS FOR VERIFIABLE MODULAR OPEN SYSTEMS OUTCOMES IN DEFENSE ACQUISITION: DESIGN, IMPLEMENTATION, AND CONTROLLED EVALUATION

A THESIS

SUBMITTED TO THE GRADUATE FACULTY
in partial fulfillment of the requirements for the
Degree of
MASTER OF SCIENCE

By DAREN JADIEL DIAZ

Tulsa, Oklahoma
2026

A CONFORMANCE HARNESS FOR VERIFIABLE MODULAR OPEN SYSTEMS OUTCOMES IN DEFENSE ACQUISITION: DESIGN, IMPLEMENTATION, AND CONTROLLED EVALUATION

THESIS COMMITTEE

GALLOGLY COLLEGE OF ENGINEERING
AND THE POLYTECHNIC INSTITUTE

Dr. John Hassell, Chair

Dr. Christopher Freeze

Dr. Teri Reed

Dr. Muhammad Sajid Riaz

Dr. Chad Roller

Revised research draft. Committee approval is pending.

© Copyright by Daren Jadiel Diaz 2026
All Rights Reserved.

Abstract

A Modular Open Systems Approach, or MOSA, aims to make systems easier to inspect, replace, and upgrade through clearly defined connections. An acquisition team still needs evidence that a proposed software component follows the declared design. This thesis develops a conformance harness, a set of repeatable checks that compares supplied evidence with explicit requirements. The revised study asks when an agent helps a person review MOSA evidence and what checks keep its recommendations reliable.

The work connects an earlier conformance experiment with a bounded agent prototype developed for an Agentic Systems capstone. The prototype examines a module description, interface contract, software component inventory, test record, and evidence history. Three controllers share the same validators and decision guard: a fixed workflow, a rule based adaptive workflow, and a model agent. The agent may choose checks and draft a recommendation. It may not alter the evidence or approve its own review. A person retains the approval decision.

The original experiment used 20 synthetic modules and showed that stronger executable checks detected all 18 seeded faults, compared with five detected by the minimal baseline. A separate capstone pilot used 20 development cases. The model matched 17 raw disposition labels; deterministic guard checks increased the match to 20. The pilot reported fewer validator selections but much longer elapsed time than the fixed workflow. These observations motivate an adaptive rule comparison and do not establish an operational advantage for artificial intelligence.

A new benchmark contains 40 neutral cases divided into development and additional evaluation sets. An offline preflight exposed a structural citation issue before hosted evaluation, and the protocol records the amendment. The study currently contains 241 recorded offline or hosted run slots out of 360 planned slots. Additional-case primary outcomes were fixed workflow: 20/20 raw and 20/20 guarded matches across 20 attempted first runs; rule based adaptive workflow: 20/20 raw and 20/20 guarded matches across 20 attempted first runs; model agent: 1/20 first runs attempted; 1 failed attempt; accuracy is not estimated for the incomplete schedule. The new hosted request failed with an unavailable resource response. Its charge could not be confirmed, so later paid requests stopped. A complete model comparison remains pending. The contribution is an inspectable review method and a reproducible offline comparison. Claims remain limited to the synthetic profile, and independent label review remains pending.

Keywords: Modular Open Systems Approach; MOSA; conformance testing; agent supported review; evidence provenance; human approval; reproducibility

Table of Contents

Chapter 1 Problem and research questions 1

Chapter 2 MOSA and evidence review foundations 4

Chapter 3 Agent design and human review 8

Chapter 4 Evaluation method 18

Chapter 5 Results and discussion 26

Chapter 6 Conclusions and future work 36

References 38

Glossary 41

Appendix A Reproduction migration and research boundaries 43

List of Figures

Figure 1. System topology showing evidence, controllers, validators, the decision guard, and the human review boundary. 9

Figure 2. A bounded review sequence from inspection to draft and guard decision. 14

Figure 3. Human review and rollback preserve source identity and review history. 17

Figure 4. Development and additional evaluation cases have distinct category counts, and the preflight amendment is disclosed. 21

Figure 5. Raw and guarded recommendation match on 20 unique additional cases; the incomplete model comparison remains unestimated. 31

Figure 6. Validation effort, elapsed time, and provider charge answer different performance questions. 33

Figure 7. A tested release moves to the thesis repository after course submission while class history remains intact. 45

Reader Guide

Start with the problem in Chapter 1, then follow the example of a sensor software replacement through Chapter 3. A reader does not need to program to understand the review. The controller decides what to inspect. Validators perform explicit checks. The decision guard checks whether the resulting recommendation follows those observations. A person decides whether to record approval.

Five colors provide navigation. Blue identifies evidence and fixed checks. Purple identifies model choices. Green means the required profile checks support readiness. Amber means information or review is still needed. Crimson marks a known blocking problem. Every color also has a text label, so the meaning remains clear in a printed copy or for a reader who cannot distinguish the colors.

Technical terms are explained near their first important use. The glossary adds an example and explains why the term matters. Chapter 4 explains the evaluation, and Chapter 5 keeps the original experiment, capstone pilot, and new benchmark separate. The public companion website provides saved walkthroughs, diagrams, data tables, and reproduction instructions. It presents research evidence; the live model and its credentials remain in a private development environment.

Label	Meaning
Blue	Evidence and fixed checks
Purple	Agent choices
Green	Supported readiness under the profile
Amber	Missing evidence or further review
Crimson	A known blocking problem

Author review and committee approval of this revision remain pending. Independent label review is also pending.

Chapter 1 Problem and research questions

1.1 Why replaceable software needs evidence

Defense software often remains in service while sensors, processors, suppliers, and threats change. A team may need to replace one component without rebuilding the whole system. The challenge is practical: the replacement must send the right information, use the expected connections, and supply enough evidence for another organization to inspect its behavior. A claim that a component is modular does not answer those questions by itself.

Consider a sensor that sends an observation to a processing service. The message includes an event identifier, a measured value, and its unit. The processor returns an alert. A supplier offers a replacement processor with the same advertised connection. If the replacement interprets a distance in feet when the sensor provides meters, the connection may work while the result is wrong. If its component inventory omits a required library, another supplier may struggle to reproduce or maintain it. These are evidence questions with consequences for replacement and review.

The Modular Open Systems Approach, abbreviated MOSA, addresses change through an integrated business and technical strategy. In plain language, it means designing systems so parts can be separated and connected through defined interfaces. An interface is the shared boundary where parts exchange information or services. The legal definition includes verification of relevant interfaces, so evidence review belongs within the approach rather than after it (10 U.S.C. § 4401, 2026).

This study makes the route from a declared requirement to a review recommendation visible and repeatable. It focuses on software evidence that a small prototype can inspect. A

reader can ask what was checked, what was missed, and who is responsible for the decision. A few files cannot establish an entire weapon system's readiness.

1.2 The gap between a claim and a supported recommendation

Public oversight gives a reason to study this gap. The U.S. Government Accountability Office, or GAO, examined 20 acquisition programs in its 2025 MOSA report. It identified weaknesses in planning, coordination, and assessment of benefits. The report also described inconsistent plans for verifying implementation at important acquisition points. These findings motivate clearer evidence practices. They do not measure this prototype or establish that an agent is the solution (GAO, 2025).

A review can go wrong in several ways. A required file may be absent. Two supplied files may disagree. A tool may fail before it checks an artifact. A reviewer may receive a confident recommendation that does not follow the recorded observations. Each situation needs a different explanation. Missing information should lead to a request for evidence. A demonstrated inconsistency should lead to a correction. A tool failure should remain visible so another reviewer can complete the inspection.

1.3 Research questions and success criteria

The central research question is: When does an agent help a person review MOSA evidence, and what checks are needed to keep its recommendations reliable? Three supporting questions make that question testable. First, do the controllers reach the expected disposition for the same unique cases? Second, how often does the decision guard correct or block a controller's draft? Third, does adaptive selection reduce validation effort enough to justify its time, cost, and added complexity?

Success means producing a review that another person can inspect and reproduce. The review must identify its evidence, distinguish known problems from unknowns, cite locations that support its findings, and preserve the person's approval role. The evaluation must compare the model with both a fixed workflow and an adaptive rule workflow. A useful result may show that explicit rules produce the same supported decisions at lower cost. The study does not require the agent to win a comparison.

1.4 Scope and contribution

The work uses constructed software evidence packages, not classified data or operational weapon systems. It tests a declared course profile with a small set of permitted validators. The profile is an explicit list of checks and input expectations. Passing it means those checks support readiness for this exercise. It does not certify compliance with every MOSA requirement, complete OpenAPI or CycloneDX standards, cybersecurity requirements, safety requirements, or contractual obligations.

The contribution has three parts. The harness turns selected requirements into repeatable observations. The agent extension preserves those checks while comparing different ways of selecting them. The public repository links the explanation to code, tests, diagrams, and saved results. Together, these artifacts let a committee member move from a plain language claim to the evidence behind it. The revision keeps the original conformance study as a foundation while concentrating the main argument on the capstone's review design and its evaluation.

Chapter 2 MOSA and evidence review foundations

2.1 Modularity and the boundary of change

A module is a component that can be described and changed as a distinct part of a system. Good boundaries reduce the amount of surrounding work needed when a design decision changes. Parnas's information hiding principle places likely changes behind a stable boundary, so other components do not need to depend on internal details (Parnas, 1972). Applied here, a sensor client should rely on the processor's published messages rather than on the processor's private source structure.

A visible connection alone is not enough. A shared message needs defined fields, units, allowed values, and expected behavior. Two suppliers can agree on a field name but disagree on its meaning. A contract that says value is a number still leaves a question about whether it represents meters, feet, seconds, or a count. The conformance harness therefore treats both structure and meaning as evidence concerns. The original prototype tested behavior as well as document presence.

Modularity also has a business side. A replacement may be technically possible but practically difficult if the customer lacks needed interface information, test access, or rights to use relevant data. The 2025 Department of Defense implementation guide connects MOSA planning with design, interfaces, verification, and acquisition considerations (Department of Defense, 2025). The present prototype addresses a selected technical evidence slice. It does not resolve licensing negotiations, intellectual property rights, or the availability of competing suppliers.

2.2 A source hierarchy for defensible claims

Different sources answer different questions. Statute establishes requirements within its stated scope. Department guidance explains implementation expectations. Oversight reports

describe observed gaps. Standards describe technical formats or procedures. Research papers supply theories and tested methods. Project files show what this prototype actually did. Keeping these roles separate prevents a strong source in one area from supporting a claim it did not examine.

Three levels of claim are kept explicit. A documented claim reports what a file or publication says. A verified observation reports what a check established under a named condition. An inferred implication explains what the observation may mean. For example, a matching interface version is a verified consistency observation. Easier supplier replacement is a possible implication that requires additional evidence. Reduced lifecycle cost is a broader outcome that this study does not measure.

2.3 What a conformance harness checks

Conformance means satisfying a stated requirement under a stated test. A harness is the surrounding code that prepares inputs, runs checks, and records results. Together, a conformance harness gives a review a repeatable procedure. The National Institute of Standards and Technology, or NIST, provides published conformance testing frameworks that connect requirements with test assertions and observable results (NIST, 2012). The thesis applies that general idea to a small MOSA evidence profile.

The capstone narrows that broad harness into three callable validators. A validator is a tool that checks explicit conditions and returns findings. The first inspects package completeness, identity, hashes, and test records. The second compares interface declarations. The third compares declared dependencies with the software component inventory and a dated advisory fixture. This smaller scope makes controller behavior easier to compare without claiming full system assurance.

A check has value when its consequence is clear. If two interface versions disagree, the review should identify both declarations and ask for alignment. If the required inventory is missing, the review should ask for the inventory and preserve uncertainty. The same green outcome should never stand for both checked agreement and unchecked absence. The three disposition categories in Chapter 3 give those situations distinct names.

2.4 Evidence identity and traceability

Evidence is useful only when a reader can connect it to the reviewed component. A manifest is a short module description that identifies the build, interface version, endpoints, and dependencies. A build is a particular produced version of the software. The test record should identify that build rather than an earlier one. A passing test record for a different build does not establish the current build's behavior.

Provenance means the recorded origin and history of an artifact. In this prototype it includes declared source information and hashes for selected files. A hash is a compact fingerprint calculated from a file's bytes. Changing the file normally changes that fingerprint. The prototype uses the Secure Hash Algorithm with a 256 bit output, written SHA 256, to bind a review to particular input files (NIST, 2015). Matching hashes support identity; they do not prove that the supplier is truthful or that the software is safe.

Traceability means a reader can follow a finding back to the evidence that supports it. A useful finding cites a file and a field location, explains the observed condition, and names a remedy. A reference to interface.json /info/version tells the reviewer where the declared interface version appears. A missing file needs a different citation: the inspected package inventory and the profile's requirement. The evaluator must recognize that absence cannot be proved by resolving a field that does not exist.

A hash identifies bytes. A hash is like a fingerprint for a file. Matching fingerprints show that the saved review and current input have the same recorded bytes. They do not show that the file is truthful or safe.

2.5 Why consider an agent and why compare it with rules

An agent in this thesis is a language model that observes tool results, chooses a permitted next check, and drafts a review. This is one use of artificial intelligence, or AI. ReAct demonstrated interleaved reasoning and information gathering actions (Yao et al., 2023). Toolformer explored model selection of external tools (Schick et al., 2023). These studies motivate tool mediated designs; they do not establish accuracy for this MOSA profile.

A rule based adaptive controller provides an essential comparison. It can inspect completeness first, follow explicit priority hints, and stop at an allowed point without invoking a language model. If the permitted decisions are simple and stable, rules may provide the same value more directly. The comparison asks what model selection adds beyond adaptive control itself. Without it, fewer checks could be attributed to intelligence when an explicit stopping rule would have achieved the same result.

Reliability has several dimensions. Correctness on one run does not show consistency across runs or useful behavior after a tool fails. Rabanser and colleagues (2026) propose assessing consistency, robustness, predictability, and safety separately when evaluating agents. This study uses that distinction to report repeated behavior, controlled failure cases, guard interventions, and cost limits alongside disposition accuracy. It does not claim to implement their entire evaluation framework.

Chapter 3 Agent design and human review

3.1 A small review system with clear responsibilities

The system separates evidence, controller, validators, decision guard, and person. Evidence enters as a read only package. The controller chooses how to inspect it. Validators return observations using named finding codes. The guard checks whether the proposed disposition follows those observations and whether the required coverage is present. The person reviews the saved report and records an approval, rejection, or request for revision.

These roles make the design easier to inspect. A model cannot hide a failed interface check by writing a reassuring summary. A validator cannot grant approval. A person cannot turn a blocking inconsistency into technical readiness by clicking approve. Recording a decision means acknowledging the review record, including its disposition and uncertainty. The distinction keeps both technical observations and human accountability visible.

The live application runs in the existing course Codespace, a development environment hosted for the class repository. Its web server presents review controls and saved records. A separate public GitHub Pages site presents static explanations and approved research exports. Static means the site delivers stored content without running the agent. A public reader can inspect a saved example without receiving access to model credentials or the private review state.

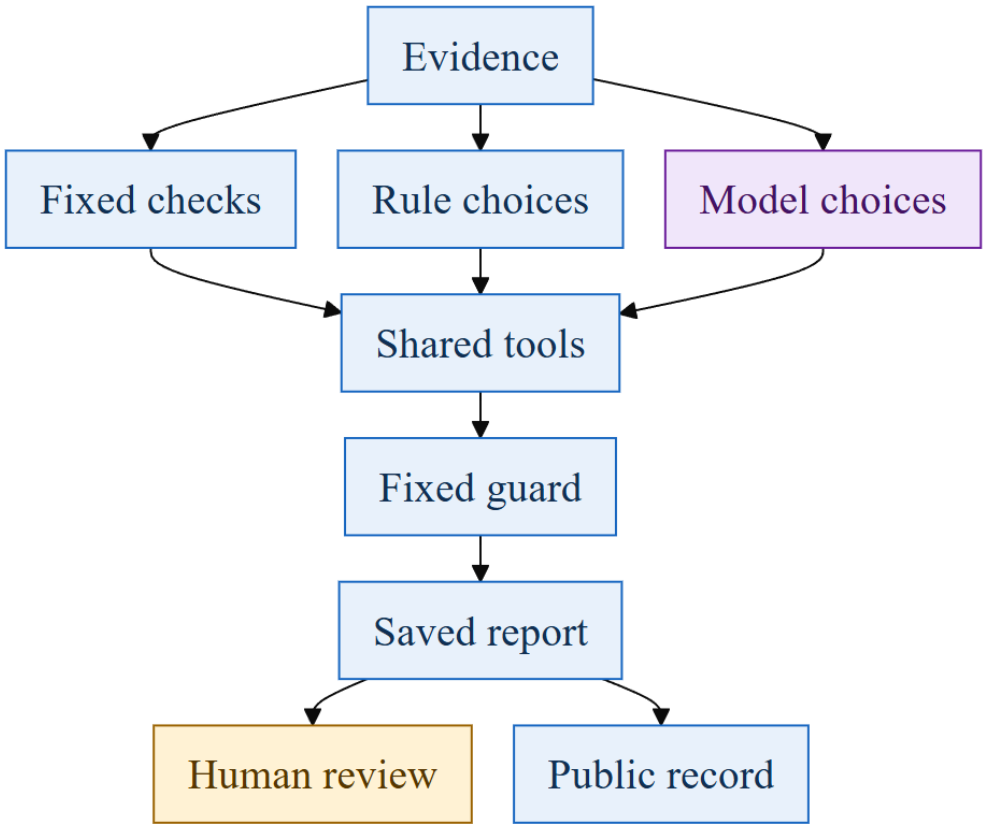


Figure 1. System topology showing evidence, controllers, validators, the decision guard, and the human review boundary.

3.2 The evidence package and its limits

Each package has five required JavaScript Object Notation files, abbreviated JSON. JSON is a structured text format that represents fields, lists, numbers, and objects in a form a computer can read (Bray, 2017). The manifest describes the module. The interface file describes its declared connections. The software bill of materials lists its components. The test file records selected outcomes. The provenance file records evidence origin and selected file fingerprints.

A software bill of materials, abbreviated SBOM, is an ingredient list for software. It helps a reviewer see which libraries and versions a component declares. The National Telecommunications and Information Administration, or NTIA, published an earlier baseline for such records (NTIA, 2021). The prototype uses selected CycloneDX fields for a defined course

exercise. CycloneDX is a published format for software and other bills of materials (OWASP Foundation, 2025). Passing these selected comparisons does not establish complete CycloneDX compliance.

The interface file uses selected OpenAPI fields. API means application programming interface, the software connection used to request data or a service. OpenAPI describes those connections in a format that people and tools can inspect (OpenAPI Initiative, 2021). The profile compares the declared version, endpoint presence, response types, and dependency declarations. An endpoint is the named route a client calls, such as a route that accepts a sensor event. The prototype does not execute the service, prove every response behavior, or validate every rule in the OpenAPI specification.

Input limits keep the exercise bounded. Artifacts must be JSON objects within the declared size limit. The package reader rejects paths that escape the package directory and rejects symbolic links, which are pointers to other filesystem locations. These checks reduce ambiguity about what was reviewed. They are specific controls with tests, not a claim that all possible hostile inputs have been ruled out. The public fixtures contain constructed evidence and no operational defense information.

3.3 Three validators shared by every controller

The package inspection tool first records which required files are available. It checks selected required fields, whether test evidence identifies the current build, and whether recorded hashes agree with current bytes. Missing evidence is marked unknown. A recorded hash that disagrees with the present file is a known inconsistency. This difference is central to a defensible disposition: the reviewer should not confuse a lack of proof with proof of a particular fault.

The interface validator compares the manifest and interface contract. It checks whether their versions agree, declared endpoints exist, selected response types match expectations, and declared dependencies remain consistent. The result names the finding, file, and relevant field. For an interface version mismatch, the remedy is to reconcile the declarations and rerun the checks. The prototype does not infer that either declaration is the correct one; that decision belongs to the supplier and reviewer.

The SBOM validator compares required dependencies against listed components and checks selected license information. It also compares declared component identities with a fixed synthetic advisory. An advisory is a notice about a known software issue. This fixture intentionally invents one advisory so the test can be repeated without relying on changing online data. It is not a real vulnerability notice. An optional live advisory lookup is outside the reported frozen benchmark unless a separate run explicitly records that setting.

Every controller uses the same validators, so tool selection is the comparison rather than inspection capability. Each actual invocation records the tool, findings, input identity, and elapsed time. Repeated invocations count as repeated work.

Table 1. The three validators and the limits of their evidence.

Validator	Question answered	Important limit
Package inspection	Are required records available and consistent with their declared identity?	Hashes support file identity, not supplier trust.
Interface comparison	Do selected interface declarations agree?	Selected fields only, not full service behavior.
SBOM comparison	Are declared components present and is the synthetic advisory matched?	Not a complete vulnerability or license assessment.

3.4 Fixed and adaptive rule workflows

The fixed workflow runs package inspection, interface comparison, and SBOM comparison in a declared order. Coverage is predictable and needs no model or pricing information. It may do unnecessary work on an incomplete package, although that work can be inexpensive for this small profile.

The adaptive rule workflow starts with package inspection. If the profile's halt condition shows that necessary evidence is missing or unusable, it records the missing requirement and returns a request for review. Otherwise, it uses an explicit priority hint to choose whether the interface or SBOM check runs next, then performs the remaining required comparison. A known failure does not excuse skipping unrelated required coverage when the evidence remains available.

The rules define which stopping points are legitimate. Early stopping is useful only when the reason is visible and the outcome preserves uncertainty. A failure in one validator may be relevant to one part of the package while other available evidence still needs inspection. The guard therefore checks the recorded halt reason and coverage rather than accepting every short trace as efficient. A short but unsupported review counts as an incomplete review.

3.5 The bounded model loop

The model agent receives the profile instructions, neutral package identifier, and allowed tool descriptions. It can request a validator, read the result, and choose another permitted step. The loop limits the number of model turns, remote retries, and output tokens. A token is a unit of text used by the provider to count input and output. The model identity and settings are stored with the run so later readers can see what produced the draft.

The agent's final draft contains a proposed disposition, the exact finding identifiers returned by tools, an explanation, and remaining uncertainties. The allowed outcomes are ready,

not_ready, and needs_review. The first means required checks support readiness within the profile. The second means observed evidence contains a blocking problem. The third means the review lacks enough supported information for a complete conclusion. The report displays plain language labels beside these stored values.

Evidence text is treated as data. A file cannot grant the model new permission, remove a required check, or tell it to approve the package. The permitted tools expose inspection actions only. Approval, source editing, and public communication are unavailable to the model. This narrow action set limits what an erroneous model instruction can do in the prototype. A prompt instruction alone is not treated as proof of safety; enforcement also occurs in the allowed tool dispatcher and guard.

A provider timeout, malformed response, unavailable model, or exhausted allowance leaves an inspectable failed or incomplete run. The application does not silently relabel a fixed workflow as a successful model run. Completed observations remain saved, and the report tells the person what remains unknown. This behavior matters because a comparison that removes failed model attempts would make the model appear more reliable than the complete record supports.

> **A model choice is a proposal.** The agent may choose a permitted check and explain its result. Fixed validators supply the observations, the guard checks support, and a person records approval.

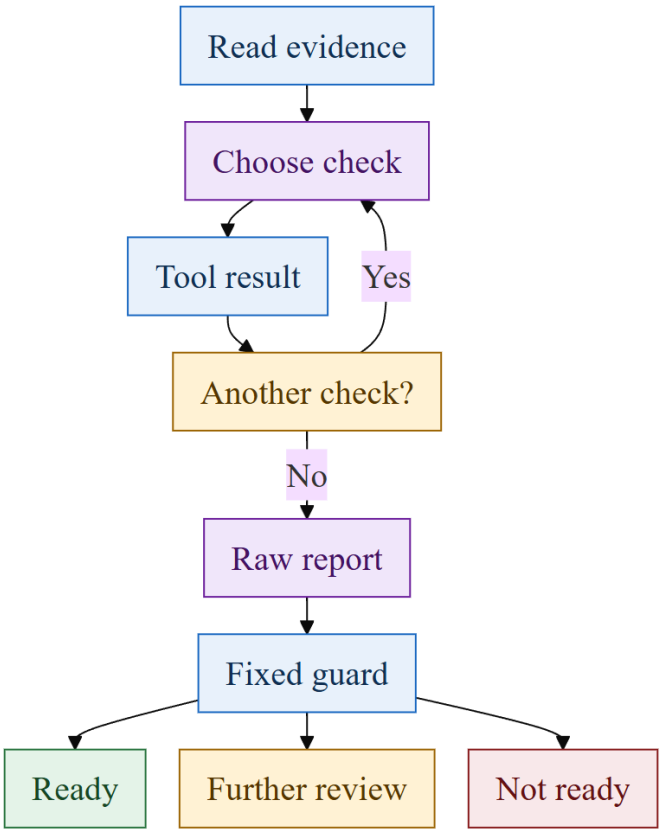


Figure 2. A bounded review sequence from inspection to draft and guard decision.

3.6 The decision guard and three possible dispositions

The decision guard is deterministic code: declared inputs and rules produce the same result. It checks known finding identifiers, required coverage or a justified halt, and support for the disposition. Unsupported or malformed drafts remain incomplete. A second model does not decide whether the first model is correct.

The guard derives the supported outcome from recorded observations. Incomplete coverage, an invalid draft, or changed source evidence requires further review. When those conditions are absent, a known blocker takes precedence over an unknown finding. If no blocker exists but evidence remains unknown, the result asks for review. Required coverage with no blocker or unknown finding supports readiness within the profile. The report preserves every finding even when one severity determines the disposition.

A guard intervention occurs when the guard corrects a disposition or blocks completion. It remains in the record. Raw and guarded outcomes show whether reliability came from the controller, fixed checks, or both. A coverage block can leave the disposition unchanged while still counting as an intervention.

A guard can enforce only rules represented in its implementation. If the profile never checks units, a matching version and component list cannot establish correct units. If a validator reports the wrong observation, the guard may faithfully follow it. The shared validators and guard make the controller comparison fair, but they also create a common source of error. Independent labels and tests are needed to challenge that shared boundary.

> **Unknown evidence is a result.** A missing test record does not establish that the component failed a test. It establishes that the reviewer lacks required evidence. The report asks for that record and explains what still needs checking.

3.7 A report a person can act on

Every report follows five questions: What was checked? What was found? Why does it matter? What should be corrected? What remains unknown? The first identifies coverage. The second names evidence based findings. The third connects the finding to replacement or maintenance. The fourth gives a practical next action. The fifth prevents the explanation from sounding more complete than the trace.

For example, a report may state that the manifest declares interface version 1.0 while the contract declares 2.0. It explains that a replacement supplier cannot rely on both declarations as the same contract. It asks the owner to select the intended version, update the related files, regenerate evidence history, and rerun review. It does not claim the component will fail in every environment or choose the correct version without additional authority.

A citation is checked at the file and field level. JSON Pointer is a standard way to address a location inside a JSON document (Bryan et al., 2013). The pointer /info/version refers to the

version field within info. The evaluator resolves such pointers for existing evidence and uses explicit inventory references for absence. It separately records whether a citation exists and whether it supports the named finding. A plausible looking string alone is insufficient.

3.8 Human approval source changes and rollback

Review output starts as a draft. A person supplies a reviewer name and explicitly selects an action. The prototype compares the current package fingerprint with the fingerprint saved at review time. If source evidence has changed, the old draft cannot be used as a current approval record. The person must rerun review or request revision. This check binds an action to the evidence actually inspected.

Approval updates a separate pointer to the recorded review. A pointer here means a stored reference to a run identifier. Rollback restores the prior approved pointer and adds a history event. It does not edit the input package or erase the newer trace. Keeping both records gives the person a way to inspect what was accepted and what was later withdrawn. The distinction is useful when demonstrating a corrected interface package after an earlier review.

The local prototype records a supplied reviewer name, not a verified organizational identity. It is not an authenticated acquisition approval system or a tamper resistant audit service. That limit is stated beside the approval demonstration. A production extension would need access control, verified identity, protected storage, and organization specific decision authority. Those are future requirements rather than capabilities inferred from the demonstration.

NIST's AI Risk Management Framework addresses context, measurement, governance, and risk management (NIST, 2023). Its generative AI profile addresses risks of generated content (NIST, 2024). This design keeps model actions bounded, preserves failures, and assigns approval to a person. It does not claim full implementation of either publication.

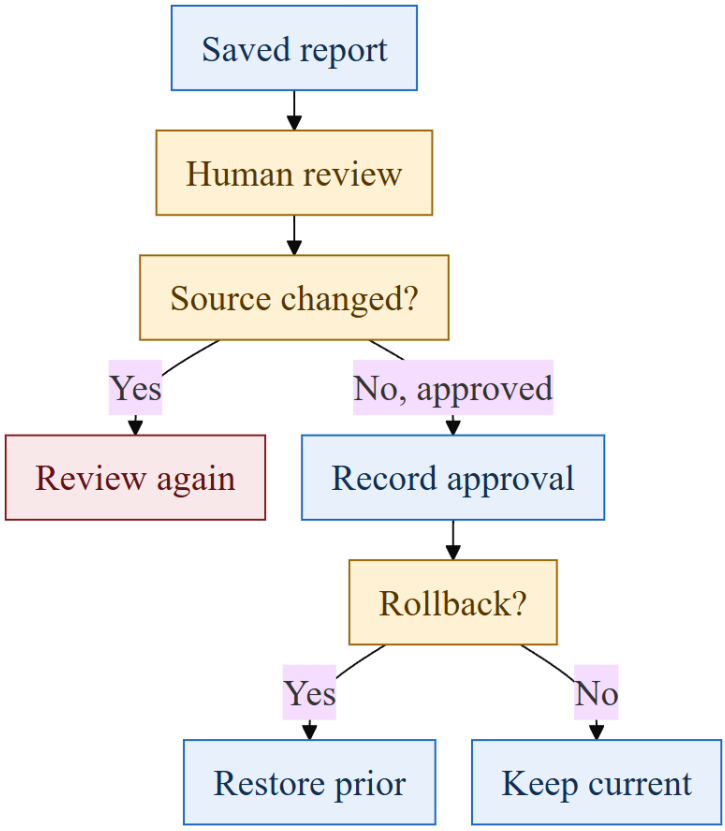


Figure 3. Human review and rollback preserve source identity and review history.

Chapter 4 Evaluation method

4.1 Design science and the unit of analysis

Design science studies a problem by building and evaluating an artifact. Hevner and colleagues (2004) describe the relation between a practical problem, a designed solution, and evidence from its use. Peffers and colleagues (2007) provide a process that moves through problem identification, objectives, development, demonstration, evaluation, and communication. This thesis follows that approach by tying each design choice to an inspectable implementation and a measured review outcome.

The artifact is the evidence review harness and its bounded controllers. The unit of analysis is one unique evidence package. A unit of analysis is the thing counted as a distinct observation when answering the research question. Running the same package three times gives information about consistency and elapsed time. It does not create three independent examples of correctness. This distinction prevents repeated execution from inflating the apparent strength of the evidence.

The study uses a controlled synthetic benchmark because the intended faults and unknowns can be specified before the controllers run. Synthetic means constructed for testing rather than collected from operational programs. Deliberately changing a valid input follows the general testing idea of mutation, which introduces a known change to challenge a test (Jia & Harman, 2011). The method makes errors observable and keeps the data safe for public sharing. Its limitation is representativeness: constructed records reflect the author's selected problem model rather than the full range of acquisition evidence or supplier behavior.

4.2 Constructing a neutral forty case benchmark

The benchmark contains 40 cases: 10 ready cases, 20 cases with known blocking problems, and 10 cases needing further review. The categories describe expected outcomes under

the profile, not universal judgments about the software. Ready cases provide all required evidence and pass selected comparisons. Blocking cases contain observable inconsistencies. Further review cases omit necessary evidence or exercise controlled uncertainty.

Each package receives a neutral identifier that does not disclose its intended condition. The earlier pilot used descriptive names that could reveal a fault to the model. Neutral identifiers remove that simple hint. Expected dispositions, required findings, and evaluation membership remain in a separate label file. They are available to the evaluator after a run, not to the model prompt or tool descriptions during review.

The ready group varies permitted evidence details without changing expected conformance. The problem group varies versions, endpoints, types, declarations, evidence identities, and component records. The unknown group includes absent records, malformed or unusable evidence where inspection cannot establish coverage, and an unavailable tool case. A case that contains malformed evidence is classified according to the published profile: a confirmed violation and an inability to complete inspection are not automatically the same condition.

A benchmark should challenge its own labels. Each case records the specific change from a ready package, the reason for the expected disposition, and the evidence location expected to support that reason. Labels are reviewed against the written profile instead of merely copying the current guard's answer. The author and assisted implementation process are disclosed. Independent review remains pending and is not described as completed simply because automated checks pass.

4.3 Development and separate evaluation sets

Twenty cases come from the earlier development fixtures, copied with neutral identifiers. They include three ready, 12 blocking, and five further review cases. Twenty additional cases include seven ready, eight blocking, and five further review cases. The sets therefore have different category proportions. Results show category counts beside overall accuracy so that a reader can assess that difference. The additional cases provide the primary reported comparison, while development results remain separate.

The initial plan aimed to hold out the additional cases from development. An offline preflight of all 40 cases exposed a structural issue in handling provenance hashes. When the hash container had the wrong shape, the validator produced a root problem and four dependent child references that were not meaningful. The amendment retains the root PROVENANCE_SHAPE blocker and suppresses those invalid dependent references. The original freeze and 240 offline preflight runs are preserved separately. No hosted model runs occurred before that amendment.

The primary dataset is therefore described as additional evaluation cases, not untouched external holdout evidence. The same research process produced the profile, fixtures, and preflight amendment. This shared origin is a limitation even when model inputs contain no labels. External cases and independent label review would provide a stronger test of generalization. Daren's verification of the labels and independent review remain pending at this revision's preparation.

The final case files, labels, prompt, model identity, controller settings, and source version are frozen in a manifest before the main evaluation. A manifest is an inventory of inputs and fingerprints. If a later correction is required, it receives a new version and explanation. The final additional set is primary and the development set has its own table. A combined total may show

run completeness, but it is not substituted for primary accuracy. This preserves the relation between development, amendment, and measurement.

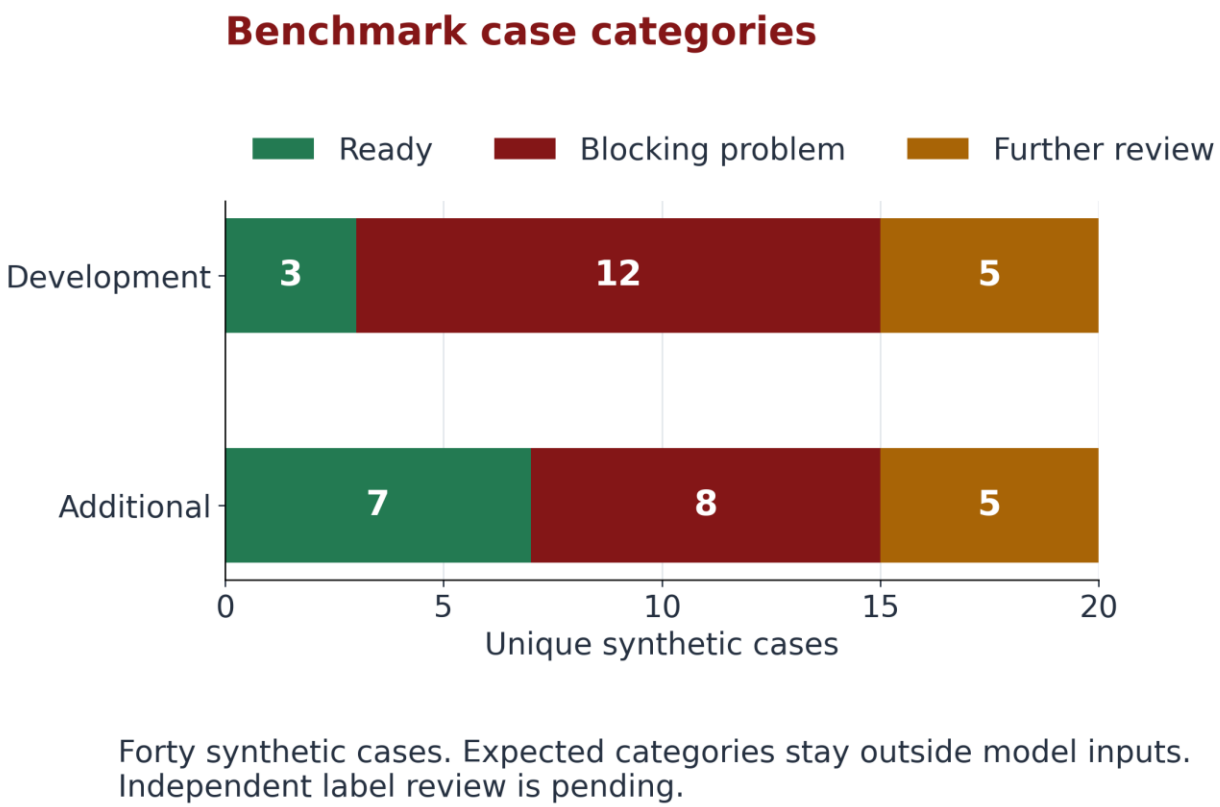


Figure 4. Development and additional evaluation cases have distinct category counts, and the preflight amendment is disclosed.

4.4 Running three approaches with repeatable settings

Each controller is scheduled to run three times on each unique package. A complete schedule therefore contains 360 runs: 40 cases multiplied by three controllers and three repetitions. Scheduling is distinct from completion. A failed or budget stopped run remains a recorded attempt, with its status and reason, rather than disappearing from the denominator. The published completion table shows what actually ran.

All controllers use the same frozen validators, evidence files, and decision guard. The model uses the existing OpenRouter configuration, with the actual model value recorded at execution. OpenRouter is the service used to access the hosted model. Remote settings include

permitted model turns, retries, response size, and timeout. Any setting changed from the capstone pilot is disclosed before interpreting a comparison across those studies.

Run order and environment are recorded. Caches and changing network or provider conditions can affect timing. Elapsed time covers the full review, including model waiting and guard work. Where available, tool time is recorded separately. The study does not attribute every timing difference to model reasoning.

The first scheduled repetition on each unique evaluation case supplies the primary accuracy comparison. All repetitions supply a stability measure and descriptive timing summaries. If the first attempt fails, it remains the primary outcome for that schedule position. A retry performed within the configured request policy remains part of the same run. A new review after a completed failure is separately identified rather than quietly replacing the failure.

4.5 Measurements that separate correctness from presentation

Raw disposition match measures whether the controller's proposed outcome equals the case label. Guarded disposition match measures whether the final supported outcome equals that label. The two measures answer different questions. If the guard repairs an incorrect draft, raw match records the error and guarded match records the recovered outcome. A report that publishes only the second would hide reliance on the guard.

Finding quality is measured by expected and returned finding identifiers. A missed finding is an expected problem absent from the report. An extra finding is a finding beyond the label's predeclared target list. It may be a grounded follow on execution error, so it is not automatically an invented problem. The trace shows whether evidence supports it. Because unknown cases can legitimately stop before later checks, expected findings are defined for the

permitted coverage of that case. A controller is not credited with an invented explanation merely because its final disposition happens to match.

Evidence reference quality checks the cited file and location against the package. Existing fields must resolve to real locations that support the named finding. Missing fields and missing files use explicit absence checks rather than nonexistent pointers. This distinction avoids giving a valid citation failure to a legitimate missing evidence report. The evaluator records citation failures so readers can inspect them case by case.

Validation effort counts actual invocation events. The previous pilot's summary counted distinct tool names held in a result mapping. If a controller called one tool twice, that summary could undercount the work. The corrected evaluator counts the event list and retains both unique tool coverage and total invocations. Coverage describes which checks occurred. Invocations describe how much work was requested. They are related, but they are not interchangeable measurements.

Elapsed time, tokens, provider charge, guard interventions, and source integrity are also recorded. Missing provider measurements remain unavailable. Offline zeros mean no hosted calls occurred; missing interactive usage cannot be interpreted as a free model run.

Table 2. Measurements and the questions they answer.

Measure	Reader question
Raw disposition match	Did the controller propose the expected outcome?
Guarded disposition match	Did the complete review support the expected outcome?
Missed and extra findings	Was the explanation complete and supported?
Evidence references	Can a reader locate the basis for the finding?
Actual tool invocations	How much validation work was requested?

Measure	Reader question
Elapsed time and provider cost	What did the review require in time and paid usage?
Guard interventions	How often did fixed rules repair or block a draft?

4.6 A five dollar ceiling with recorded stopping

The new hosted evaluation has a total spending ceiling of five U.S. dollars. The cost ledger records planned reservations, completed provider usage, and unresolved charges. Before a request, the runner estimates a conservative maximum charge from the known input size, response limit, and reliable provider prices. It reserves that allowance before sending the request. A request is refused if the reservation would exceed the remaining ceiling.

A reservation must bound all permitted billable work. The text only evaluation rejects search, plugins, multimodal content, and unknown request options. Verified token and request prices plus provider price caps define the bound. Unpriced reasoning or other usage causes a stop. Actual charges reconcile reservations; unresolved billing after a timeout does not immediately release the allowance.

This limit is part of the experimental constraint, not evidence that the model is economical in general. The earlier pilot's low observed charge cannot guarantee that a larger study will cost the same amount. Provider pricing or behavior may change. If reliable price information cannot be obtained, the study records the hosted comparison as incomplete and still preserves the offline results. Incomplete evidence is a legitimate study outcome, provided it is explained accurately.

4.7 Analysis and limits of inference

Accuracy is reported as a numerator and denominator for unique cases, with category results beside the overall result. A result states how many of the seven ready additional cases

received a ready recommendation, how many of the eight blocking cases were correctly identified, and how many of the five uncertain cases remained requests for review. This lets the reader see whether an apparently strong total hides confusion about a particular disposition.

A confidence interval describes uncertainty around a proportion under an assumed sampling model. It is not a numerical confidence assigned to an individual agent recommendation. Wilson score intervals are appropriate for small proportions and extreme observed counts (Wilson, 1927; Newcombe, 1998). In this synthetic study they summarize finite case results under the stated model; they do not transform constructed cases into a representative sample of defense programs.

Paired comparisons use the same unique cases under different controllers. The exact McNemar test compares the numbers of cases where one approach is correct and the other is not (McNemar, 1947). If exploratory comparisons are shown for all controller pairs, their descriptive status is stated and raw counts remain primary. Repetitions are used for stability and timing rather than entered as independent correctness observations. With only 20 primary evaluation cases, interpretation should emphasize observed differences and uncertainty.

> **Repeated runs are not new cases.** Three reviews of one package show how consistent the workflow is. They still count as one distinct package when estimating correctness across cases.

Chapter 5 Results and discussion

5.1 Three evidence sets with different meanings

Results are presented in three layers. The original conformance experiment establishes the value of stronger executable checks for its seeded fault model. The capstone pilot shows how a model selected tools and how the guard handled its decisions. The new benchmark provides the primary controller comparison for this revision. Keeping the layers separate prevents a result produced under one profile from being presented as validation of another.

The public result tables link each published measure to its source record. A saved run includes evidence identity, controller settings, tool events, proposed disposition, guard output, and completion status. Charts are generated from the same verified export used for the tables. A committee member can therefore follow a summary result to a case outcome and then to the supporting trace. Any unavailable measure remains identified as unavailable.

Table 3. The evidence sets are distinct and are not pooled.

Evidence set	Unique inputs	Main purpose	Main limit
Original conformance experiment	20 modules with 18 seeded faults	Observe faults through stronger checks	Constructed fault model and two controls
Capstone pilot	20 development evidence packages	Demonstrate bounded model review	Descriptive names and one hosted sweep
New benchmark	40 neutral packages with 20 separate evaluation cases	Compare three controllers	Synthetic profile and pending independent label review

5.2 The original experiment as a foundation

The original minimal baseline detected five of 18 seeded nonconformances. The conformance harness detected all 18. Both accepted the two conformant controls. Fault detection

was therefore 27.8 percent for the minimal baseline and 100 percent for the harness on that constructed set. The respective Wilson 95 percent intervals were approximately 12.5 to 50.9 percent and 82.4 to 100 percent. The upper result should be read as complete detection of the selected faults, not proof that every possible fault would be found.

Thirteen unique packages were correct under the harness and incorrect under the minimal baseline; none favored the baseline. The exact two sided McNemar result was $p = 0.000244$. A p value describes how unusual the paired difference would be under the test's equal correctness assumption. It does not measure the probability that MOSA works, nor does it quantify an operational program's likelihood of success. The constructed fault set determines the scope of this result.

Both controls passed, but two controls offer little information about false positives in a larger population. Their 100 percent observed acceptance has a wide Wilson interval of approximately 34.2 to 100 percent. This matters because a harness that detects seeded problems may still reject legitimate supplier variation not represented by the controls. More diverse conformant packages would strengthen that part of the evidence.

The archive also records greater computational effort. Its primary result fields contain median durations of about 0.348 milliseconds for the minimal baseline and 7.042 milliseconds for the harness, with median check counts of 14 and 177. The completed thesis contains other rounded timing summaries, so this revision names the archived primary fields rather than mixing timing populations. The inference remains narrow: more checks bought more observed evidence in a small local experiment.

5.3 The capstone pilot and guard corrections

The fixed course workflow matched all 20 expected disposition labels in the pilot. The standalone OpenRouter model matched 17 of 20 raw labels, and the guarded outcome matched all 20. Expected fault localization was recorded for all 17 faulted cases. The pilot thus demonstrates that the guard materially affected the final result. It does not justify describing the unguarded agent as fully correct.

The raw model called three missing evidence cases not ready: missing manifest, missing SBOM, and missing provenance. The profile expected further review because required evidence was absent. The guard changed those recommendations to needs_review. This correction illustrates a practical distinction. A missing ingredient list means the reviewer cannot inspect the required component declaration. It does not establish that every declared or undeclared component is defective.

Another saved pilot trace showed a failed SBOM validator followed by skipped interface comparison. The guard blocked completion because required coverage was incomplete. This fourth intervention concerned the path through the checks rather than a simple label correction. Together, the examples show that guard value depends on both disposition rules and coverage rules. The final label alone would not reveal the failure mode.

The archived pilot summary reports 51 model selected validator calls compared with 60 for the fixed workflow. The model made 71 remote calls, used 42,016 input tokens and 2,354 output tokens, and reported a provider charge of $0.0205728. Median elapsed time was 3,916.684 milliseconds for the model and 4.98 milliseconds for the fixed workflow in the same Codespace. Reduced recorded validator selections therefore did not produce faster reviews.

Those call totals come from the pilot's earlier evaluator, which counted distinct tool names in each saved result mapping. They describe that archived export and should not be

treated as a reliable general measure of repeated invocation effort. The new evaluator uses event counts. The pilot also used descriptive case names and one sweep rather than a blinded repeated design. Its results motivate the new benchmark and are not substituted for it.

5.4 Interactive demonstrations are a separate kind of evidence

Six selected Codex traces demonstrated different validator order and an early stop for missing evidence. They matched the expected labels for those selected examples and recorded 16 validator selections compared with 18 for the same fixed reviews. These traces are useful for showing the file bridge and bounded workflow. They are not an independently blinded model accuracy study.

The interactive controller had surrounding developer context that the standalone OpenRouter prompt did not necessarily receive. Its token usage was not available through the prototype's remote provider counters. A recorded zero in those counters therefore does not establish that the interactive model used no tokens or incurred no underlying cost. The public explanation keeps that missing measurement visible.

Replaying saved selections demonstrates those steps, not a new autonomous decision. The public walkthroughs likewise present recorded traces. Expanding or animating a saved review does not perform another model evaluation.

5.5 Primary results from the separate evaluation cases

The completion record identifies what actually ran. The final study records 241 of 360 planned run slots. The comparison is incomplete, and unattempted slots are not counted as executed evidence. The provider returned code 404 for the hosted Hypertext Transfer Protocol request, abbreviated HTTP. This indicates that a requested resource was not found. No reliable

charge was supplied, so later paid requests stopped. Failed and unattempted slots retain their status. Offline results cannot supply missing hosted measurements.

The first scheduled repetition on the 20 unique additional cases defines primary accuracy. For the 20 additional evaluation cases, fixed workflow: 20/20 raw and 20/20 guarded matches across 20 attempted first runs; rule based adaptive workflow: 20/20 raw and 20/20 guarded matches across 20 attempted first runs; model agent: 1/20 first runs attempted; 1 failed attempt; accuracy is not estimated for the incomplete schedule. The first run of each case is the accuracy unit; the other repetitions do not increase the sample size. Raw, guarded, and category results show both controller decisions and reliance on the guard.

fixed workflow had 0 missed seeded finding identifiers, 1 extra finding identifiers, and 15/15 valid recorded references in its primary runs; rule based adaptive workflow had 0 missed seeded finding identifiers, 0 extra finding identifiers, and 14/14 valid recorded references in its primary runs. Extra fixed-workflow findings can be follow-on validator failures when evidence is missing; they are reported separately from seeded root faults. Missing target findings, additional findings, and invalid references remain distinct measures. A citation must resolve to supporting content or satisfy an explicit absence check.

Development results remain separate. In the 20 development cases, fixed workflow: 20/20 raw and 20/20 guarded matches across 20 attempted first runs; rule based adaptive workflow: 20/20 raw and 20/20 guarded matches across 20 attempted first runs; model agent: not measured yet. These development cases copy the earlier checkpoint evidence under neutral identifiers. Agreement on cases used to refine the implementation supplies a useful development record, while shared synthetic construction limits generalization in both sets.

Table 4. Primary disposition results from the first scheduled repetition of unique evaluation cases.

Controller	Raw match	Guarded match	Completed repeated runs
Fixed	20/20	20/20	60/60
Adaptive rules	20/20	20/20	60/60
Model agent	Not estimated	Not estimated	0/60

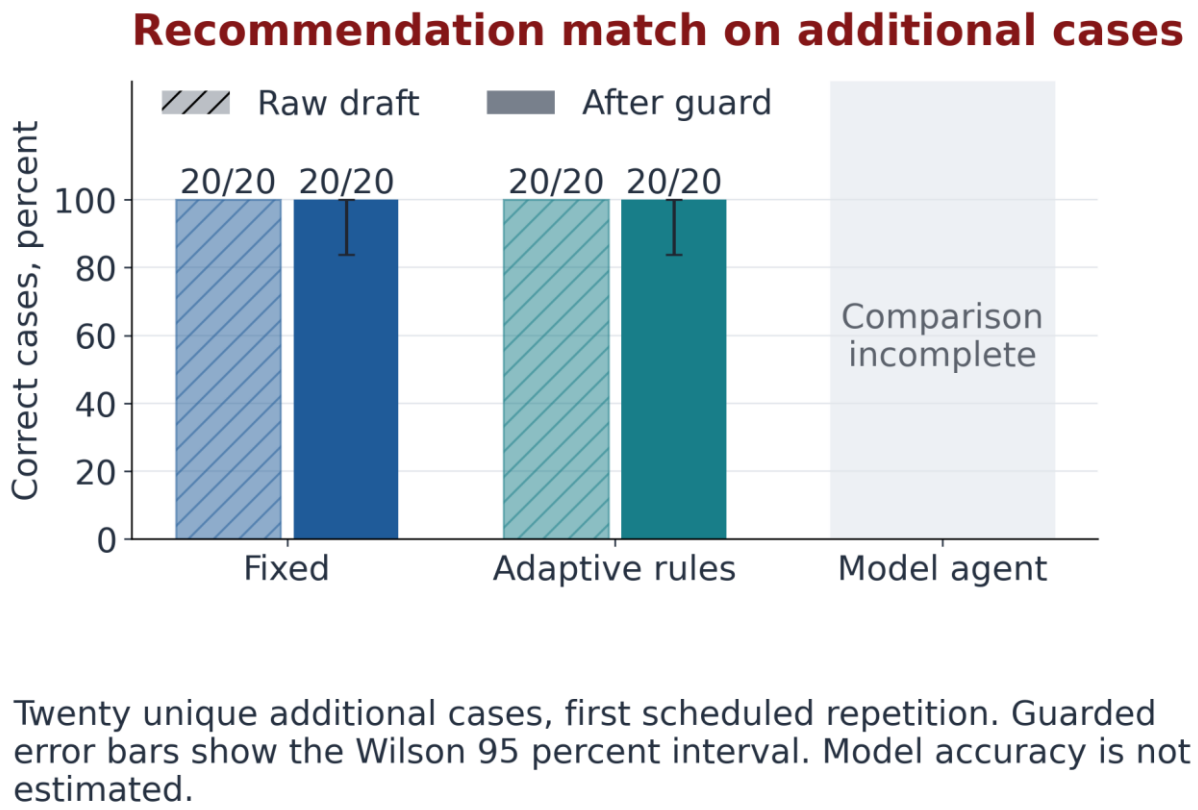


Figure 5. Raw and guarded recommendation match on 20 unique additional cases; the incomplete model comparison remains unestimated.

5.6 Effort elapsed time cost and stability

Effort counts actual invocation events. Across the three repetitions on the 20 additional cases, fixed workflow: 180 actual validator invocations across 60 recorded runs; rule based adaptive workflow: 162 actual validator invocations across 60 recorded runs. Incomplete model

attempts do not provide a comparable validation-effort estimate. Required coverage and a justified halt determine whether reduced work supports a complete review.

For completed additional-case reviews, fixed workflow: median 5.009 milliseconds and $0 in reported provider charges; rule based adaptive workflow: median 4.969 milliseconds and $0 in reported provider charges. These numbers describe the recorded execution environment and exclude human review time. An incomplete or failed model attempt is not used as a successful-review timing or cost estimate. Hosted time includes network and provider waiting. The recorded provider charge is not a complete organizational cost estimate.

On additional cases with all three repetitions, fixed workflow: 20/20 stable raw recommendations and 20/20 stable guarded recommendations; rule based adaptive workflow: 20/20 stable raw recommendations and 20/20 stable guarded recommendations. No model stability estimate is made without completed repetitions. Raw and guarded stability remain separate. A stable final result can hide changing drafts repaired by the guard. Failed and stopped attempts remain visible.

The ledger contains $0 in reported provider charges and $0.00936056 held under conservative accounting against the $5 ceiling. The actual charge is unknown. Paid work stopped because request failed and its provider charge is unknown. The reported amount is not evidence of zero spending. A refused request can demonstrate a working spending control, but it cannot supply the empirical result of the request that did not run.

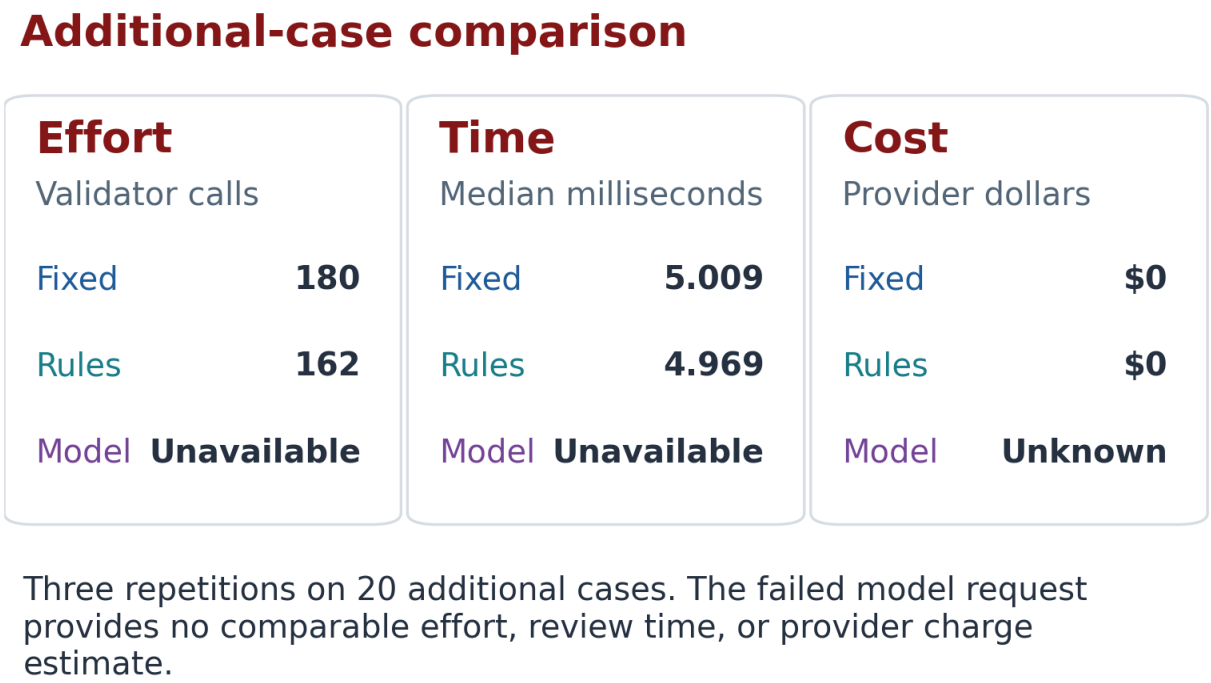


Figure 6. Validation effort, elapsed time, and provider charge answer different performance questions.

5.7 What the comparison can establish

The pilot motivates comparison with an adaptive rule alternative. The completed offline evidence supports an adaptive rule comparison, but the incomplete model schedule prevents a complete three-controller empirical conclusion. Independent review remains pending, and these cases cannot establish performance in a real acquisition setting. Conclusions follow the primary records rather than assuming that model selected checks add value.

If adaptive rules match guarded accuracy while using fewer checks than the fixed workflow, the study supports selective control for this profile. If the model achieves the same outcome while adding latency and cost, the additional model layer needs another measured benefit to justify it. Potential benefits such as clearer explanations or reduced human effort would require their own evaluation with readers; this benchmark does not measure them by intuition.

If the model identifies different useful checks or improves a measured outcome, the report should show which cases produced that difference and whether it survived the guard. A model's plausible explanation is not a substitute for a supported finding. Likewise, an accuracy tie does not establish equal behavior when guard interventions, missed findings, citations, and failures differ. The design keeps those differences visible.

5.8 Limitations and the next evidence needed

The study's inputs are small synthetic records. They do not include the ambiguity, document volume, varied suppliers, or changing operational conditions of a full acquisition review. The same author directed the profile, cases, and implementation. Independent label review remains pending. A public repository improves inspectability, but inspectability does not by itself make the evaluation independent.

The shared validators are another boundary. A defect in a common validator can affect every controller and still produce an apparent tie. Tests cover declared behaviors and selected failure paths; they do not prove the absence of every implementation defect. The guard prevents unsupported dispositions only within its encoded rules. It does not validate requirements that the profile omits.

Human review remains demonstrated rather than measured as a user study. The project shows approval, revision requests, source change checks, and rollback, but it does not quantify committee comprehension or time saved for acquisition staff. It also does not establish an authenticated approval chain. A later study could ask independent reviewers to interpret saved reports, locate evidence, and choose a justified action before comparing their effort across report formats.

The strongest next evidence would come from independently labeled packages, additional conformant variation, and realistic but releasable supplier examples. New checks could test executed message semantics and broader component inventories. Any extension should keep versioned profiles and repeat the controller comparison, because a model's value may depend on how complex the permitted decisions become. The present study gives a reproducible starting point for that investigation.

Chapter 6 Conclusions and future work

6.1 What this thesis contributes

MOSA review needs a clear connection between a claim, supplied evidence, and the decision a person records. This thesis develops that connection through a conformance harness and a bounded agent extension. The original experiment supports stronger executable checks within its seeded fault model. The capstone pilot shows why model drafts and guard outcomes must remain separate. The new benchmark provides an offline controller comparison and records a hosted stopping condition; it does not complete a new three controller comparison.

The completed offline evidence supports an adaptive rule comparison, but the incomplete model schedule prevents a complete three-controller empirical conclusion. The conclusion does not require a model advantage. If a rule based workflow achieves the same supported outcomes with lower time or cost, that result provides a useful design choice. If the hosted evaluation remains incomplete, the conclusion reports that limit and relies only on the completed evidence. A failed or stopped run is part of the research record.

The most defensible role for the agent in this implementation is assistance with inspection choices and a draft explanation. Required observations remain in validator outputs, the guard checks the supported disposition, and the person retains approval. These boundaries make the prototype reviewable. They do not establish official certification or unrestricted readiness for defense use.

6.2 A focused path for further work

Further work should strengthen the evidence before expanding the system's authority. Independent label review is the first priority. Additional packages should vary legitimate interfaces and supplier practices, because detecting designed faults does not establish a low false

rejection rate. A reader study could then measure whether plain language reports help people find evidence and understand uncertainty without concealing technical detail.

Technical extensions can add executed semantic tests, broader standards validation, live advisory snapshots, and stronger identity controls. Each addition needs a defined profile, expected outcomes, and a recorded comparison. A larger document search agent would require separate evaluation of retrieval and citation support. Multiple agents or additional model providers should be introduced only when a concrete evidence need justifies their extra complexity.

The class repository and its checkpoint history remain intact. After course submission, a tested release can be copied to the thesis repository with its source commit, hashes, and evaluation configuration. The public reading site will remain a static guide to the research, while live review remains in a private environment. This separation allows the committee to inspect the work easily and lets later thesis development proceed with a clear source history.

References

10 U.S.C. § 4401. (2026). Requirement for modular open system approach in major defense acquisition programs; definitions. Office of the Law Revision Counsel, U.S. House of Representatives. Source

Bray, T. (Ed.). (2017). The JavaScript Object Notation (JSON) data interchange format (RFC 8259). Internet Engineering Task Force. Source

Bryan, P., Zyp, K., & Nottingham, M. (2013). JavaScript Object Notation (JSON) Pointer (RFC 6901). Internet Engineering Task Force. Source

Department of Defense. (2025). Implementing a Modular Open Systems Approach in Department of Defense programs. Office of the Under Secretary of Defense for Research and Engineering. Source

Diaz, D. J. (2026a). A conformance harness for verifiable modular open systems outcomes in defense acquisition: Design, implementation, and controlled evaluation [Completed research draft and accompanying executable archive]. University of Oklahoma. Source

Diaz, D. J. (2026b). MOSA evidence review agent: Capstone design review and progress checkpoint [Software prototype and recorded pilot evidence]. Agentic Systems course. Source

Diaz, D. J. (2026c). MOSA evidence review thesis: Frozen benchmark, evaluation protocol, and recorded controller results [Research software and dataset]. Source

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. MIS Quarterly, 28(1), 75 to 105. Source

Jia, Y., & Harman, M. (2011). An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering, 37(5), 649 to 678. Source

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, 12(2), 153 to 157. Source

National Institute of Standards and Technology. (2012). Conformance testing methodology for ANSI/NIST ITL 1 2011, data format for the interchange of fingerprint, facial and other biometric information, Release 1.0 (NIST SP 500 295; F. L. Podio, D. Yaga, & C. J. McGinnis, Eds.). Source

National Institute of Standards and Technology. (2015). Secure hash standard (Federal Information Processing Standards Publication 180 4). Source

National Institute of Standards and Technology. (2023). Artificial intelligence risk management framework (AI RMF 1.0) (NIST AI 100 1). Source

National Institute of Standards and Technology. (2024). Artificial intelligence risk management framework: Generative artificial intelligence profile (NIST AI 600 1). Source

National Telecommunications and Information Administration. (2021). The minimum elements for a software bill of materials (SBOM). U.S. Department of Commerce. Source

Newcombe, R. G. (1998). Two sided confidence intervals for the single proportion: Comparison of seven methods. Statistics in Medicine, 17(8), 857 to 872. Source

OpenAPI Initiative. (2021). OpenAPI specification, version 3.1.0. Source

OWASP Foundation. (2025). CycloneDX specification, version 1.7. Source

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. Communications of the ACM, 15(12), 1053 to 1058. Source

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. Journal of Management Information Systems, 24(3), 45 to 77. Source

Rabanser, S., Kapoor, S., Kirgis, P., Liu, K., Utpala, S., & Narayanan, A. (2026). Towards a science of AI agent reliability [Preprint]. arXiv. Source

Schick, T., Dwivedi Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. Advances in Neural Information Processing Systems, 36. Source

U.S. Government Accountability Office. (2025). Weapon systems acquisition: DOD needs better planning to attain benefits of modular open systems (GAO 25 106931). Source

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association, 22(158), 209 to 212. Source

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations. Source

Glossary

MOSA. Modular Open Systems Approach. A business and technical strategy for changing separable system parts through defined interfaces. Example: Replace a processor while keeping an agreed sensor message. Why it matters: The thesis tests a small evidence slice of this broader approach.

Agent. A model that observes results, selects permitted actions, and drafts a response. Example: Choose interface comparison after package inspection. Why it matters: Its selection and draft are evaluated separately from guard output.

Adaptive workflow. A process whose next step follows an explicit condition found during review. Example: Stop and request a missing manifest. Why it matters: Rules can provide selective effort without a model.

Artifact. A file or record used as evidence. Example: An interface contract or saved test result. Why it matters: A review needs identifiable inputs.

Conformance. Satisfaction of a declared requirement under a declared check. Example: Two files declare the same interface version. Why it matters: A passing check has a bounded meaning.

Decision guard. Deterministic code that checks coverage and the support for a proposed disposition. Example: Convert a missing evidence draft to needs_review. Why it matters: A correct final result may depend on guard correction.

Disposition. The technical outcome recorded for a review. Example: Ready, not ready, or further review needed. Why it matters: It is distinct from human approval.

Interface. The defined boundary through which components exchange data or services. Example: A message from a sensor to a processor. Why it matters: Replacement requires a dependable boundary.

JSON. JavaScript Object Notation. A structured text format for fields, objects, and lists. Example: A version field stored inside an info object. Why it matters: Validators can read the evidence consistently.

JSON Pointer. A standard field address inside a JSON document. Example: /info/version addresses the version inside info. Why it matters: A reader can find the support for a finding.

SBOM. Software bill of materials. An inventory of software components and related details. Example: A processor lists a library and its version. Why it matters: A reviewer can compare declared dependencies with supplied ingredients.

Provenance. Recorded origin and history of evidence. Example: A source identifier and hashes for produced files. Why it matters: A review should apply to the evidence actually inspected.

Hash. A fingerprint calculated from file bytes. Example: A new file produces a different SHA 256 fingerprint. Why it matters: Detects changes but does not prove supplier trust.

Unknown evidence. Information that is absent or could not be established. Example: A required validator is unavailable. Why it matters: It cannot be described as a passed check.

Blocker. An observed condition that violates a required profile check. Example: Manifest and interface versions disagree. Why it matters: The report names a correction before readiness.

Held out case. An evaluation input kept separate from controller development. Example: A neutral package first run after settings are frozen. Why it matters: Reduces tuning on reported cases but does not create independent authorship.

Latency. Elapsed time before the complete review result is available. Example: A model waits for a remote provider response. Why it matters: Fewer checks may still take longer.

Token. A provider counted unit of model input or output text. Example: An evidence message contributes input tokens. Why it matters: Helps account for paid model usage.

Confidence interval. A statistical range describing uncertainty around a measured proportion under stated assumptions. Example: Two accepted controls produce a wide interval. Why it matters: It is not the model's personal confidence score.

Rollback. Restoring a previous approval reference while retaining history. Example: Return to an earlier accepted review record. Why it matters: It changes the current pointer without erasing evidence.

Appendix A Reproduction migration and research boundaries

A.1 Begin with a saved example

The companion site's Start Here page links to the thesis, a saved review, the comparison results, the glossary, and reproduction instructions. Begin with a ready case and a missing evidence case. Compare the proposed disposition with the guard output. Then open the file and field cited by one finding. A reader can inspect this route without running a model or entering credentials.

The repository separates original evidence, pilot evidence, and the new benchmark. Each has its own manifest or saved source record. The chart data comes from the verified result export, so the same numerator and denominator appear in the thesis, README, and site. README means the introductory repository document. It explains the folder map and directs readers to a suitable level of detail.

A.2 Run offline checks before hosted evaluation

A fresh checkout should first run the documented offline tests and fixed and adaptive evaluations. Use the Python version stated by the tested release. Python is the programming language used by the prototype. No hosted credential is needed for the offline controllers. Compare the resulting logical outcomes with the published case records. Local timings may differ on another machine and should not be required to match byte for byte.

Hosted reproduction requires a privately configured OpenRouter credential and a new explicit spending allowance. A credential is the secret that authorizes use of the provider account. Keep it outside tracked files, screenshots, public logs, and the website. The five dollar ceiling in the reported study applies to that recorded experiment; it is not automatically a continuing allowance for future reruns. The recorded model and settings should be copied exactly when investigating repeatability.

A.3 Preserve a change record

If a case, label, prompt, validator, or guard changes, save a new study version and identify the reason. Preserve the earlier records. Update diagrams and prose when they describe altered behavior, then regenerate tables and charts from the new result export. A version label alone is insufficient if the underlying inputs cannot be recovered. The frozen manifest provides the fingerprints needed to inspect that relationship.

A.4 Copy a tested release after course submission

Keep the class repository, its original checkpoint files, and its course history intact. After submission, select a tested capstone commit. Copy only the approved prototype release, public fixtures, tests, and releasable evaluation exports to the thesis repository. Record the source commit, relative file map, SHA 256 hashes, profile version, and evaluation settings. Exclude secrets, private review state, temporary service addresses, and unrelated coursework.

Test the copied release from a fresh checkout of the thesis repository. It must run without importing code from the class folder. Compare the logical outcomes and execute the offline tests. Preserve author and inherited license notices. After that verification, the thesis repository becomes the source for later thesis work, while the class repository remains the historical course submission. The migration guide supplies exact commands for the tested release rather than asking the reader to infer which files are safe to publish.

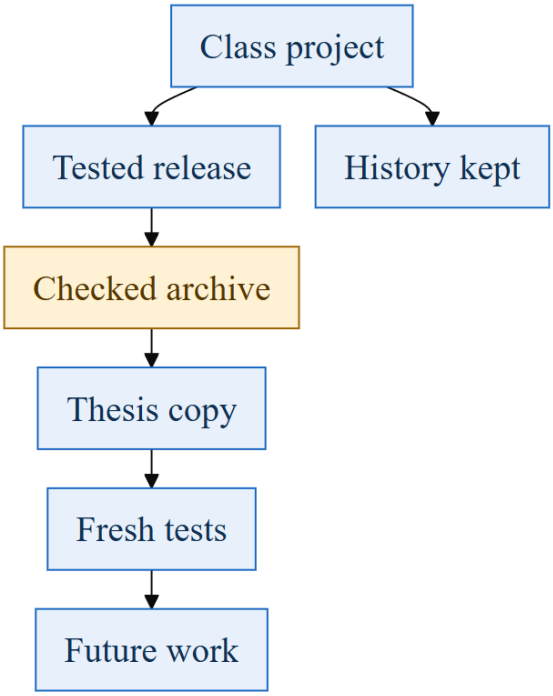


Figure 7. A tested release moves to the thesis repository after course submission while class history remains intact.

A.5 What the public site does and does not claim

The public website hosts stored research explanations and evidence. It does not run the model, grant approval authority, or provide a certification service. Its readiness labels refer to the synthetic profile. The evidence packages are constructed examples, and the advisory fixture is explicitly synthetic. Independent label review and committee approval of the revision remain pending unless later dated records establish them.

Topics such as full acquisition governance, financial forecasting, legal rights negotiation, organization wide rollout, and domain specific mission certification remain outside this condensed study. They may affect a real implementation, but this prototype does not measure them. Model based systems engineering, or MBSE, is the use of structured models to support system design. Connecting such models to this profile is a possible extension rather than a demonstrated feature of the present evaluation.